

PBSMS

Internal Engineering Copilot

Engineering Process Specification · Version 1.0 · Model B of two AI specifications · Internal — not a product feature

Item	Value
Document	PBSMS Internal Engineering Copilot — Engineering Process Specification
Version	1.0
Status	For review. Not approved for adoption. No tooling selected; see §13.
Classification	Internal engineering process. Not an SRS chapter. Creates no system requirement. Tenants are never aware of it.
Requirement prefix	EC- (Engineering Copilot). These are process rules binding the team, not software requirements binding the product.
Owner	Whoever holds the Engineering Lead role
Companion	Model A — Tenant AI Assistant, SRS v2.1 Chapter 47. A product feature. Shares nothing with this document except the word "AI".

1. What This Is, and What It Is Not

The Internal Engineering Copilot is an AI coding assistant operating inside the PBSMS repository, used by the PBSMS engineering team. It clusters user feedback into a ranked backlog, answers questions about the codebase, detects gaps between the SRS and the implementation, drafts change proposals, and implements approved ones as pull requests on feature branches.

It is not deployed in the product. It has no tenant. It has no user outside the engineering team. It ships nothing by itself.

This document is separate from Chapter 47 for a specific reason: the original proposal treated both as one system, and treating them as one system is the principal risk this separation exists to remove. Model A is a feature a school pays for and uses. Model B is a tool the engineering team uses to build the thing the school pays for. They have different data, different threat models, different approvers, and no shared infrastructure.

1.1 Hard separation from the product

- **EC-001** — The Copilot shall not be deployed in, callable from, or reachable by the PBSMS production runtime.
- **EC-002** — The Copilot shall hold no credentials to any production database, no production API keys, and no path to any tenant record.
- **EC-003** — The Copilot and the Tenant AI Assistant shall share no infrastructure, no credentials and no code path. A compromise of one shall not reach the other.

- **EC-004** — No output of the Copilot shall reach a tenant except as reviewed, merged, tested, released code — on exactly the same path as any human engineer's work, with no shortcut and no separate channel.
- **EC-005** — The Copilot shall not modify Chapter 47 artefacts — the Assistant's prompts, scope configuration, retrieval rules or evaluation gates. This is a prohibition, not a review requirement. A tool that can edit the constraints on another AI system is a tool with a much larger blast radius than its purpose justifies.

2. Capabilities

ID	Capability	Output
EC-100	Feedback aggregation	Deduplicated, clustered themes from in-app feedback, support tickets and telemetry friction signals
EC-101	Impact ranking	A ranked backlog with request volume, affected roles, affected tenant count and severity — an input to prioritisation, never prioritisation itself
EC-102	Repository question answering	Answers about the codebase with file and line references, for engineers
EC-103	Change proposal drafting	An RFC: problem, affected SRS requirement IDs, proposed change, schema impact, tenant impact, rollback plan, test plan
EC-104	Implementation	A pull request on a feature branch, with tests and a migration where needed
EC-105	Test authoring	Tests covering its own changes, subject to §5
EC-106	Documentation drift detection	A report of where a merged change has made the SRS or the Frontend Design Specification inaccurate, plus a drafted amendment. It does not amend either document.
EC-107	Gap detection	Requirements in the SRS with no corresponding implementation, no test, or both — reported, not fixed

EC-107 is the "find and fix gaps" capability from the original proposal, split in two. Finding is automated and safe: it is a comparison between two documents the Copilot can read. Fixing is not automated, because deciding whether an unimplemented requirement should be implemented, deferred, or deleted from the SRS is a product judgement, and roughly a third of the time the correct fix is to change the requirement rather than the code. Gaps go to the backlog. The backlog goes to a person.

3. The Merge Boundary

- **EC-200** — The Copilot shall never merge to main. There is no maturity threshold, no accumulated success rate, and no trusted-mode flag that unlocks this.

The original proposal was that the Copilot be trained repeatedly, and once its success rate was high enough, be permitted to push and merge to main. Three specific things are wrong with that, in order of seriousness.

3.1 Self-referential validation proves the wrong thing

A change the Copilot wrote, covered by tests the Copilot wrote, passing a suite the Copilot can influence, demonstrates that the Copilot is internally consistent. It demonstrates nothing about whether an RLS policy still isolates one school's records from another, whether a published result is still immutable under FR-RES-030, or whether a fee proration rule still matches a Ghanaian three-term year. Those are questions about intent, and intent is not recoverable from the code that was supposed to express it. This is why §5 makes certain test suites human-authored and immutable to the Copilot — they are the only part of the pipeline that is not downstream of the Copilot's own understanding.

3.2 Accountability is a legal requirement, not a cultural preference

Ghana's Data Protection Act requires an identifiable controller accountable for decisions about how personal data is processed. PBSMS processes the personal data of children. A merge that changes how that data is stored, exposed, retained or shared is a processing decision and needs a name against it. "The model merged it" is not a position available to a data controller under audit, and the point at which it becomes unavailable is the point at which the incident has already happened.

3.3 Success rate is the wrong instrument

A 99% success rate over a thousand merged changes is ten defects shipped into a shared-database, shared-schema system holding minors' records and school finances. The useful question is not how often the tool is right; it is what happens on the occasion it is wrong. In this architecture the tail risk is cross-tenant data exposure, which is silent, unbounded in scope, not detected by the tenant, and not undone by a rollback — the data has already been seen. A metric that averages that event with a thousand successful button-colour changes is not measuring the thing that matters.

3.4 The rules that follow from this

- **EC-201** — Every Copilot pull request requires review and approval by a named human engineer before merge, on the same terms as a human-authored pull request.
- **EC-202** — Pull requests touching a protected zone (§5) require two human approvers, one of whom holds the Engineering Lead role.
- **EC-203** — Deployment to production is human-triggered. The Copilot shall hold no deploy permission.
- **EC-204** — The Copilot shall not approve any pull request, including one it did not author.
- **EC-205** — The Copilot shall not modify CI configuration, branch protection rules, or repository permissions.

4. Feedback Pipeline

Source	Signal	Handling
In-app feedback	Free-text submission from any surface, with role and screen context	Anonymised at ingestion (EC-300)
Support tickets	Tenant-raised issues and requests	Anonymised; tenant identity replaced by an opaque identifier
Telemetry	Abandonment, repeated retries,	Recorded against role and screen, never

Source	Signal	Handling
	dead-end navigation, validation-error clustering	against an identified user
Assistant error reports	Answers reported as incorrect under FR-AIT-602	Question text is personal data — stripped before it reaches the Copilot; only the failure class crosses
Tenant success conversations	Notes from calls with schools	Summarised by a human before ingestion, never raw

- **EC-300** — Anonymisation at ingestion. Student names, guardian names, staff names, contact details, admission numbers, invoice numbers, BECE index numbers and any free-text identifier shall be stripped before feedback reaches the Copilot's context. The Copilot works on the shape of a problem, not on who reported it.
- **EC-301** — Tenant attribution shall be by opaque identifier only. The Copilot may know that fourteen distinct tenants reported the same friction. It shall not know which fourteen.
- **EC-302** — No production data in the Copilot's context, at any time, in any form — not a sample, not an export, not a temporary copy for debugging. Where a defect cannot be diagnosed without real data, a human diagnoses it.
- **EC-303** — Telemetry shall be behavioural, not personal: events against role and screen, with no user identifier and no tenant identifier beyond the opaque one.

4.1 Cross-tenant contamination

This is the failure mode specific to a shared-schema multi-tenant product, and it is not a security failure — it is a product failure that looks like a feature.

- **EC-310** — Volume is not a mandate. A change requested by many tenants does not thereby become a change every tenant receives. The ranked backlog is an input to a decision made by a named human product owner, who is accountable for that decision.
- **EC-311** — Shared schema means everyone inherits. Any merged change altering a workflow a tenant depends on shall ship behind a feature flag with tenant-level opt-in, defaulting to existing behaviour. The default protects the tenants who did not ask.
- **EC-312** — No mid-term workflow changes. Changes to attendance, assessment, results or invoicing workflows shall not be released to a tenant during an active term without that tenant's explicit consent. A Ghanaian school year has three terms; a workflow change landing in week six of one is disruption, not improvement, however well-requested it was.
- **EC-313** — A minority tenant shall not be outvoted into breakage. Where a highly-requested change would degrade the workflow of tenants who did not request it, the flag default holds them on existing behaviour until they choose otherwise.

5. Protected Zones and Immutable Suites

Some parts of this codebase are load-bearing for correctness properties that cannot be recovered after they break. The Copilot may draft changes in most of them, under stricter review. In two of them it may not act at all.

Zone	Why	Rule
RLS policies; any migration touching a tenant-owned table	The entire tenant isolation boundary	May draft. Two approvers incl. Engineering Lead. RLS coverage gate and cross-tenant isolation suite must pass.
Tenant context middleware, AsyncLocalStorage scoping, request-scoped DB service	Carries the isolation boundary into every request	Same as above
Authentication, authorisation, Chapter 13 scope resolution	Access control for minors' records	Same as above
Finance ledger, allocation, reversal (Ch 23–25)	Money correctness; maker-checker integrity	May draft. Two approvers, one holding finance domain ownership.
Results immutability and publication gate (FR-RES-020, FR-RES-030)	Academic record integrity; BECE and CSSPS depend on it	Same as above
Subscription metering and billing (TEN-030, TEN-031)	Revenue correctness	Same as above
Chapter 47 Assistant prompts, scope config, evaluation gates	Constraints on another AI system	No access. Prohibited outright (EC-005).
Protected test suites (below)	The only validation not downstream of the Copilot	No access. Prohibited outright (EC-400).

- **EC-400** — The following suites shall be human-authored and immutable to the Copilot: the cross-tenant isolation suite (NFR-QA-020), the Chapter 47 Assistant isolation and grounding gates, the finance invariant suite, and the results-immutability suite. The Copilot may propose additional cases in a pull request that touches nothing else; it may never modify or delete an existing case.

This is the single most important rule in the document. A tool that can edit the tests constraining it is not constrained. Every other safeguard here degrades gracefully under pressure; this one does not degrade at all, which is why it is expressed as an access prohibition enforced by repository permissions and a CI check, rather than as a review convention.

- **EC-401** — Copilot-authored tests are evidence, not proof. A reviewer shall assume that such a test encodes the Copilot's understanding of the change — the same understanding that produced the change. Review shall check the test against the requirement, not against the code it accompanies.

6. CI Gates for Copilot Pull Requests

A Copilot pull request passes every gate below before a human is asked to look at it. Spending a reviewer's attention on a change that an automated check would have rejected is the fastest way to make reviewers stop reading carefully, and reviewer attention is the actual safety mechanism in this whole design.

ID	Gate	Fails when
----	------	------------

ID	Gate	Fails when
—	Existing pipeline	SAST, dependency scan, RLS coverage gate, cross-tenant isolation suite, accessibility gate (pa11y-ci), performance budget — all as already configured
EC-500	Protected-path check	The diff touches a protected zone without the required label and approver count
EC-501	Immutable-suite check	Any protected test file is modified or deleted
EC-502	Migration safety check	A destructive migration appears without an explicit, separately reviewed exception
EC-503	Diff ceiling	The change exceeds the reviewable size limit (proposed 400 changed lines) and has not been split
EC-504	Traceability check	The pull request does not reference the SRS requirement IDs it implements or changes
EC-505	Behaviour-change flag check	A user-visible workflow change ships without a feature flag (EC-311)
EC-506	Attribution check	The commit and pull request are not labelled as Copilot-authored

7. Change Workflow

1	Signal	feedback, tickets, telemetry, gap detection	COPILOT
2	Cluster & rank	deduplicated themes, ranked backlog	COPILOT
3	Product decision	what gets built, and for whom	HUMAN
4	RFC draft	problem, impact, schema, rollback, tests	COPILOT
5	RFC approval	technical approach signed off	HUMAN
6	Implementation	feature branch, code, tests, migration	COPILOT
7	CI gates	§6 – all must pass before review is requested	AUTOMATED
8	Code review	against the requirement, not the code	HUMAN
9	Merge	the accountability record	HUMAN
10	Release	flagged, default off, human-triggered	HUMAN
11	Rollout	tenant opt-in, staged, never mid-term	HUMAN
12	Measurement	adoption, defects, reverts	AUTOMATED
13	Doc drift	SRS / Frontend Spec amendment drafted	COPILOT
14	Doc approval	amendment accepted into the baseline	HUMAN

Humans act at seven of the fourteen stages, and at every stage where something becomes irreversible or reaches a tenant. That is the design, not a transitional arrangement pending greater confidence in the tool.

8. Accountability

Role	Accountable for
Engineering Lead	This document; protected-zone approvals; the decision to advance a rollout stage; invoking EC-700
Product Owner	Stage 3 product decisions; which tenants receive which flagged change and when
Reviewing engineer	The correctness of the merged change. Named in the commit trail as author of record for audit and DPA purposes.

Role	Accountable for
Data Protection Officer	That EC-300 anonymisation holds in practice; that no processing decision reaches production without a named human behind it

- **EC-600** — Every merged Copilot change shall carry the reviewing engineer's name in the commit trail. That engineer is the author of record.
- **EC-601** — Copilot authorship shall be labelled in the pull request and the commit, so the proportion of AI-authored code in the system is measurable rather than folkloric.
- **EC-602** — An incident caused by a Copilot-authored change shall be reviewed as a failure of the gates and the process, not of the reviewer alone. A process that depends on a reviewer never having a bad afternoon is not a process.

9. Metrics

Metric	Why it is tracked	What a bad number means
Review rejection rate	Whether review is real	Very low is the warning sign, not the good sign — it suggests rubber-stamping rather than quality
Time-to-review	Reviewer load	Rising means reviewers are saturated and review quality is falling before defect rates show it
Defect escape rate, Copilot vs human PRs	The core comparison	Copilot exceeding human baseline triggers EC-700
Share of PRs touching protected zones	Blast-radius exposure	Should stay low. Rising means the tool is being pointed at the wrong work.
Reverts within 30 days	Change quality	Rising indicates the gates are passing changes that should not pass
Signal-to-ship latency	The actual benefit being claimed	If this does not fall, the tool is not delivering the thing it was adopted for

- **EC-700** — Suspension condition. If the defect escape rate of Copilot-authored merged changes exceeds the human baseline over two consecutive quarters, or if any Copilot-authored change causes a cross-tenant data exposure, implementation capability (EC-104, EC-105) shall be suspended immediately and only the analysis capabilities (EC-100 to EC-103, EC-107) retained pending review.

Naming the shutdown condition in advance is what makes it possible to actually invoke. A condition agreed after an incident is a negotiation; a condition agreed before one is a rule.

10. Failure Modes

Failure	How it shows up	Containment
Plausible but wrong implementation	Passes self-authored tests, fails intent	EC-401 review posture; protected suites (EC-400); requirement-referenced review

Failure	How it shows up	Containment
		(EC-504)
Reviewer fatigue	Approval rate rises, review time falls	Diff ceiling (EC-503); metrics in §9; this is the hardest failure to detect early and has no good leading indicator
Scope creep in a single PR	A small fix carries an unrelated refactor	Diff ceiling; traceability check; reviewer rejects rather than untangles
Feedback poisoning	A tenant submits feedback crafted to steer the backlog	EC-300 anonymisation; EC-310 human product decision; volume is an input, not an instruction
Protected-zone drift	Zone boundaries erode as the codebase grows	Zone definitions reviewed each quarter by the Engineering Lead; new isolation-critical code added to the list when written, not later
Silent dependency on the tool	Team loses the ability to work without it	Rollout stages are reversible by design; EC-700 suspension must remain survivable
Cross-tenant exposure from a merged change	Possibly never detected by the tenant	The reason for two approvers, immutable isolation suites, and EC-700 naming this as a single-instance trigger

11. Rollout Stages

Stage	Capability enabled	Advance when
1	Feedback clustering and repository Q&A. No pull requests.	The backlog it produces is demonstrably more useful than a product owner reading tickets — which is currently an assumption, not a finding.
2	Gap detection (EC-107), reporting only	Reported gaps are real gaps at an acceptable rate; false-positive rate is tolerable to the team
3	Test-only pull requests — added coverage, no production code touched	Lowest-risk way to calibrate review posture and measure review quality before anything can break
4	Implementation PRs outside protected zones, small diffs, flagged	A human defect-escape baseline exists to compare against (see §13)
5	Wider implementation, still outside protected zones	Stage 4 defect escape rate at or below human baseline for two quarters
6	Protected zones, drafting only, two approvers	A distinct decision by the Engineering Lead, reviewed on its own merits. Not automatic on the success of stages 1–5.

Stage 3 before stage 4 is deliberate. Test-only pull requests let the team find out how carefully it actually reviews Copilot output at a point where a missed defect costs nothing. Discovering the answer to that question at stage 5 is considerably more expensive.

12. On Training, and on Success Rates

Three points, directly on the original proposal, because they change what should be built rather than only what should be permitted.

- You do not train a model on the repository. Retrieval over the current codebase is both cheaper and more correct: a fine-tuned model encodes the repository as it was on the day of training and drifts silently as the code changes, with no signal that it has. Retrieval reads what is there now. The improvement path for this tool is better retrieval, better prompts, better evaluation cases and better gates — not model weights.
- "Success rate" needs a denominator that means something. Measured against self-authored tests, it measures internal consistency. Measured against human review acceptance, it measures usefulness — worth tracking, and it is the review rejection rate in §9. Neither measures safety. Safety is measured only by the suites the Copilot cannot touch, which is the entire reason EC-400 exists.
- The merge button is not a bottleneck to be optimised away. It is the accountability record. Removing it does not remove the risk; it removes the name attached to the risk, which under Ghana's Data Protection Act is the one thing that cannot be missing.

13. Open Questions

- No tooling has been selected. The requirements that constrain the choice are EC-002 (no production credentials), EC-400 (enforceable file-level access restrictions) and EC-506 (attribution), and they should be checked against candidates before stage 1, not after.
- The 400-line diff ceiling in EC-503 is a guess. Re-set it from stage 3 measurement of how review quality actually degrades with diff size on this team, which is the only place that number can honestly come from.
- No baseline defect escape rate exists for human-authored PBSMS changes, so EC-700's comparison currently has no denominator. Establishing it is a prerequisite for stage 4 and it takes at least a quarter of ordinary development to produce.
- Reviewer fatigue is the real failure mode of this entire design and it has no good leading indicator. The metrics in §9 detect it late. If someone finds a better early signal, that is a more valuable contribution to this document than any other change to it.
- Cost per merged change is unmodelled, so the economic case for adoption is unquantified.
- Whether automated feedback clustering produces a better backlog than a product owner reading tickets directly is an assumption. Stage 1 exists specifically to test it, and stage 1 failing is an acceptable outcome that saves the cost of stages 2 through 6.
- This document, like Chapter 47, has been written but not reviewed by anyone other than its author.

— End of Document —